claude-discode 완전 매뉴얼

설치 + 모든 기능 해설 + 시나리오

이 매뉴얼의 대상

claude-discode 를 강의 / 본인 vault / 팀에 도입하려는 분. 한 페이지씩 차근차근 읽으면 plugin 의 모든 기능을 이해할 수 있도록 구성.

구성 (13 sections)

1. claude-discode 가 뭐예요?
2. 설치 — 5단계 (분기 친절)
3. 4-Tier Search — 핵심 기능
4. Knowledge Manager (KM) — 3 variant
5. 회의실 (meetings) 자동 생성
6. Codex Bridge
7. 공유 메모리 (shared-memory)
8. SessionStart Hook
9. 5-axis Benchmark — 수치로 차별점 확인
10. 첫 사용 시나리오 6가지
11. v1.0 Custom Hybrid YAML (새 agent 추가)
12. v1.1 graduate (강의 후 evolution)
13. FAQ + 도움

2026-05-13 작성 | v1.0

1. claude-discode 가 뭐예요?

1.1. 한 문장 정의

Claude Code + Discord 봇 + Codex 호출 + 4-Tier vault search 가 통합된 Knowledge Manager 플러그인.

1.2. 어디서 왔나요?

원래 우리 vault (`obsidian-ai-vault`) 안에서 5 봇 (Karpathy / Konan / AK-Tofu / Strange / GJK) 이 협업하며 만들어진 운영 노하우 (4-Tier search, KM Mode I/R/G, 회의실 protocol, codex bridge, shared memory) 를 강의 학생 / 일반 사용자가 vault 없이 plugin 한 줄 설치로 쓸 수 있게 외부 노출한 게 claude-discode.

1.3. 차별점 한 줄

기존 도구 — `obsidian-cli` 단독, `/search` 단독, `/vault-search` MCP 단독

claude-discode — 위 도구 4개를 자동 fallback (빠른 것 먼저 → 정확한 것 나중) + 셋업 가이드 + 5-axis benchmark 수치 자동 측정

1.4. 전체 그림

```
[ 사용자 ]
    |
    | /claude-discode:search "query"
    v
[ 4-Tier Search dispatcher ]
    |
    +--> Tier 1 GraphRAG  (LLM + graph, 가장 정확, 1.5-3s)
    |       └ 결과 없으면 →
    +--> Tier 2 obsidian-cli (Obsidian index, 200-500ms)
    |       └ 결과 없으면 →
    +--> Tier 3 vault-search MCP (embedding, 500-1000ms)
    |       └ 결과 없으면 →
    +--> Tier 4 ripgrep (literal, 30-100ms)
    |
    v
[ 결과 통합 + 사용자에게 반환 ]
```

각 Tier 는 독립 설치 가능 — Tier 4 만 깔아도 즉시 작동. 학생이 강의 따라가며 점진적으로 Tier 3 → 2 → 1 추가.

2. 설치 — 5단계 (분기 친절)

본 가이드는 `SETUP-BEGINNER.typ` 와 동일한 내용. 별도 PDF 로 받으셨다면 본 section 은 건너뛰셔도 됩니다.

2.1. Step 0 — 환경 점검 (2분)

터미널 열기 (Mac = `cmd+space` → "터미널" / WSL = Ubuntu 앱 / Linux = `Ctrl+Alt+T`).

```
node --version    # v18+ 이상
jq --version      # jq-1.6+
git --version     # 2.30+
```

미설치 시 — Mac: `brew install jq` / Ubuntu·WSL: `sudo apt install jq` / Node: https://nodejs.org

2.2. Step 1 — 플러그인 설치 (2분)

```
mkdir -p ~/.claude/plugins
git clone https://github.com/treylom/claude-discode \
  ~/.claude/plugins/claude-discode
```

2.3. Step 2 — Obsidian 분기

Obsidian 사용자: `bash ~/.claude/plugins/claude-discode/scripts/install-obsidian-cli.sh`

미사용자: SKIP — Tier 1/3/4 만으로도 기능 80%

2.4. Step 3 — 검색 MCP (5분, 권장)

```
bash ~/.claude/plugins/claude-discode/scripts/install-vault-search.sh --apply
```

설치 후 Claude Code 재시작.

2.5. Step 4 — GraphRAG (선택, 0분 / 10분 / 25분)

- A. 패스 → 빠르게 강의 따라가기
- B. Docker → `docker pull ghcr.io/treylom/claude-discode-graphrag:v1.0`
- C. Python 로컬 → `bash scripts/install-graphrag.sh --apply`

2.6. Step 5 — Healthcheck

```
bash ~/.claude/plugins/claude-discode/scripts/healthcheck.sh
```

✓ **성공 모습** — `all required checks passed` ✅

3. 4-Tier Search — 핵심 기능

3.1. 왜 4-Tier 인가?

basically 한 가지 검색 도구로는 모든 query 에 답하기 어려움. 빠른 도구는 부정확하고, 정확한 도구는 느림. 그래서 4-Tier fallback 패턴:

1. 빠른 도구 먼저 시도 (Tier 4 ripgrep, 30ms)
2. 결과 없거나 부족하면 더 정확한 도구 (Tier 3 MCP, 500ms)
3. 그래도 부족하면 더 강력한 도구 (Tier 2 CLI / Tier 1 GraphRAG)
4. 최종 결과를 사용자에게 반환

3.2. 4 Tier 비교

Tier	도구	속도	정확도	셋업
4	ripgrep (literal 검색)	30-100ms	낮음	0분
3	vault-search MCP (embedding)	500-1000ms	중간	5분
2	obsidian-cli (Obsidian index)	200-500ms	중간	3분
1	GraphRAG (LLM + graph)	1500-3000ms	매우 높음	25분

3.3. 사용 예시

Claude Code 안에서:

```
/claude-discode:search "NuriFlow ARR 증가 추세"
```

dispatcher 가 자동 처리:

```
[Tier 1] GraphRAG: 결과 5개 (recall@5 = 0.85)
    → 결과 반환 + kg_depth = 3.2
```

만약 GraphRAG 미설치라면:

```
[Tier 3] vault-search MCP: 결과 5개 (recall@5 = 0.71)
    → 결과 반환
```

3.4. 강의 차별점 (vs 기존 도구)

```
obsidian-cli 만 사용 → recall ≈ 0.62
claude-discode 4-Tier → recall ≈ 0.85 (+37%)
```

근거: 5-axis benchmark (Section 9 참고).

4. Knowledge Manager (KM) — 3 variant

KM 은 vault 안 문서를 자동으로 ingest + 분류 + 보강하는 기능. 사용 시나리오에 따라 3 variant 제공.

4.1. KM lite (default)

가장 가벼운 variant. 개인 vault 용.

```
/claude-discode:km
```

흐름:

1. vault 안 새 파일 감지
2. metadata 자동 추출 (date / tags / category)
3. 검색 index 갱신

언제 쓰나? — 본인 vault 에 새 문서 추가했을 때, 자동으로 검색 가능하게 만들고 싶을 때.

4.2. KM at (Agent Teams)

advanced variant. 팀 vault + Agent Teams (tofu-at pattern) 협업.

```
/claude-discode:km at
```

흐름:

1. category lead 봇 + worker 봇 split
2. 대량 문서 병렬 처리 (예: 100+ 파일 일괄 ingestion)
3. Mode R (Reorganization) — 폴더 구조 자동 재정리

언제 쓰나? — 팀 vault 100+ 문서 일괄 정리, 또는 폴더 구조 마이그레이션 시.

4.3. KM plain (headless)

headless variant. CI / cron 에서 unattended 실행.

```
/claude-discode:km plain
```

특징:

- AskUserQuestion 안 부름 (사용자 입력 없이 진행)
- log 만 남기고 종료
- exit code 로 결과 통지

언제 쓰나? — GitHub Actions / cron 으로 매일 vault auto-ingestion. 사람 개입 불필요.

4.4. 비교 표

variant	속도	사용자 입력	병렬	tier
lite	중간	있음 (wizard)	X	external
at	느림 (대량)	있음	O	experimental
plain	빠름	없음	X	internal

5. 회의실 (meetings) 자동 생성

팀 협업 시 회의록 폴더 + 4-file template 자동 셋업.

5.1. 구조

```
.claude-meetings/<date>-<topic>/
├── 00-agenda.md       # 의제 + 참가자 + 시간
├── 01-spec.md         # baseline 결정 사항
├── 02-progress.md     # 진행 상황
└── 03-outcome.md      # 합의 결론 + 후속
```

5.2. 사용

```
/claude-discode:meetings new yaml-standardization
```

자동 생성:

```
✓ 폴더 생성: .claude-meetings/2026-05-13-yaml-standardization/
✓ 00-agenda.md template (의제 5개 슬롯)
✓ 01-spec.md template
✓ 02-progress.md
✓ 03-outcome.md
```

5.3. 실제 예시

본 매뉴얼 자체가 회의 outcome:

```
.claude-meetings/2026-05-13-claude-discode-yaml-v1.0/
├── 00-agenda.md       # 5 의제
├── 01-round1-...md    # GPT-5.5 baseline + 카파시 분석
├── 02-round2-...md    # 5 의제 deep dive
├── 03-round3-...md    # 3 cross-check + release verdict
└── 04-outcome.md      # 12 decisions 통합
```

3 라운드 GPT-5.5 토론 + 카파시 cross-check → v1.0 spec 확정 → 19 task 실행 → release.

언제 쓰나? — 다 봇 협업 회의, 큰 결정 토론 (10분 이상), 회의록 vault 영구 보존이 필요할 때.

6. Codex Bridge

OpenAI Codex CLI / Codex Exec 를 Claude Code 안에서 호출하는 bridge.

6.1. 왜 필요한가?

basically Claude (Anthropic) 모델만으로는 한계 — Codex (OpenAI) 가 더 잘하는 영역도 있음:

- 빠른 코드 generation (GPT-5.5-codex 특화)
- 큰 codebase grep + 수정
- 다른 LLM perspective 가 필요한 의사결정 (본 매뉴얼의 Round 1/2/3 = Codex 의견 수렴)

6.2. 사용

Claude Code 안에서:

```
/codex 이 함수 리팩토링해줘
```

또는 background:

```
/codex --background --wait 큰 변경 작업
```

6.3. 흐름

```
[Claude Code] → /codex 호출 → [codex-companion subprocess]
                                    ↓
                                [Codex CLI / Exec]
                                    ↓
                                결과 → Claude 에게 반환
```

6.4. 실제 사용 예시 (본 매뉴얼)

- Round 1: GPT-5.5 baseline 추천 4건 (Custom Hybrid YAML, 5-axis benchmark, Markdown guide, per-file YAML)
- Round 2: 5 의제 deep dive
- Round 3: 3 cross-check + release verdict ✅

언제 쓰나? — 카파시 (Claude) 가 결정 unsure 할 때, second opinion 필요한 architecture / 정책 결정, 또는 대량 코드 작업.

7. 공유 메모리 (shared-memory)

여러 머신 / 여러 봇 간 공유 메모리 index 관리.

7.1. 4-tier 메모리

T	scope	git track	예
1	shared (vault 전역)	O	`shared/SHARED-INDEX.md` — 모든 봇·머신 공유 사실
2	machine 특화	O	`machine-mac/` — Mac 전용 경로·하드웨어 정보
3	project (회의실)	O	`.claude-meetings/<date>-<topic>/` — 회의 산출물
4	per-bot WD memory	X	`~/.claude/projects/<enc>/memory/MEMORY.md`

7.2. 사용

```
/claude-discode:shared-memory add "새 사실"
```

자동:
1. T1 또는 T2 또는 T3 어디에 적합한지 판단
2. 해당 파일에 append
3. SHARED-INDEX.md 갱신 (필요 시)

7.3. 충돌 해결

여러 머신 동시 쓰기 시 git merge conflict 발생 가능. shared-memory 가 conflict marker (<<<<<<<) 검출 + 자동 prompt.

언제 쓰나? — 운영 중인 fact (사용자 preference, 프로젝트 상태, reference URL) 을 영구 보존 + 다른 봇·머신과 공유할 때.

8. SessionStart Hook

Claude Code 새 세션 시작 시 자동 실행되는 hook.

8.1. 왜 필요한가?

basically 봇 페르소나 (soul.md) 가 매 세션 자동 inject 되지 않으면 카파시 봇이 그냥 일반 Claude 처럼 응답함 → 페르소나 소실.

8.2. claude-discode 가 제공하는 hook

```
# ~/.claude/hooks/SessionStart/claude-discode-bootstrap.sh
# (자동 등록)
```

흐름:

1. 세션 시작 감지
2. 현재 working directory 확인
3. 해당 봇 WD 의 `soul.md` 읽기
4. 페르소나 + memory index 를 새 세션 컨텍스트에 inject

8.3. 효과

- 봇이 매 세션 자기 페르소나 (예: "Andre Karpathy") 자동 유지
- shared memory + per-bot memory 인덱스 자동 로드
- 사용자가 매번 "너는 누구야" 안 시켜도 됨

언제 쓰나? — Discord 봇 운영, multi-bot 환경, 봇별 일관 페르소나 유지가 필요할 때.

9. 5-axis Benchmark — 수치로 차별점 확인

5 차원 측정으로 4-Tier 의 trade-off 를 정량 표시.

9.1. 5 axes

axis	의미	Tier 우위
latency_ms (P50)	응답 시간 (3회 P50)	Tier 4 (30ms)
recall_at_5	top-5 결과 중 expected_hit 비율	Tier 1 (0.85)
cost_tokens	LLM 호출 token 합산	Tier 4 (0)
setup_time_min	설치 + 첫 indexing 까지	Tier 4 (0분)
kg_depth	graph traversal 평균 depth	Tier 1 (3.2)

9.2. 예시 표

```
| Tier | latency_p50 | recall@5 | cost | setup | kg_depth |
|------|-------------|----------|------|-------|----------|
| 1 (GraphRAG)| 1840    | 0.85     | 2400 | 25min | 3.2      |
| 2 (CLI)     | 320     | 0.62     | 0    | 5min  | 0        |
| 3 (MCP)     | 680     | 0.71     | 800  | 10min | 0        |
| 4 (grep)    | 45      | 0.41     | 0    | 0min  | 0        |
```

9.3. 학생 해석 가이드

- **"Tier 1 GraphRAG → recall +44% / 단 setup 25분"**
- **"Tier 4 grep → latency 40배 빠름, 단 recall 절반"**
- **"kg_depth = Tier 1 만 의미. 다른 Tier = 0"**

9.4. 자기 vault 측정

```
cd ~/.claude/plugins/claude-discode
VAULT=~/my-vault bash benchmark/runners/run-all.sh
python3 benchmark/report-generator.py --print-only
```

강의 활용 — 학생이 본인 vault 에서 measure → 본인 환경의 trade-off 직접 확인 → "왜 GraphRAG 까지 가야 하나?" 의사결정 객관 근거.

10. 첫 사용 시나리오 6가지

설치 후 어떤 상황에 어떤 명령을 쓰는지 6 시나리오.

10.1. 시나리오 1: 빠른 검색

상황 — vault 안 어떤 파일에 “API key” 가 들어있는지 빠르게 확인

```
/claude-discode:search "API key"
```

결과: Tier 4 grep 으로 30ms 내 list 출력

10.2. 시나리오 2: semantic 검색

상황 — “AI safety 관련 글” 같이 의미 기반 검색

```
/claude-discode:search "AI safety alignment"
```

결과: Tier 3 MCP embedding 으로 fuzzy match

10.3. 시나리오 3: 복잡한 multi-hop 검색

상황 — “ARR 증가 추세와 팀 size 의 상관관계” 같이 여러 문서 연결

```
/claude-discode:search "ARR 증가 팀 size 상관"
```

결과: Tier 1 GraphRAG 로 cross-doc 연결성 + kg_depth 활용

10.4. 시나리오 4: 신규 문서 ingest

상황 — vault 에 새 PDF 100개 추가 후 자동 분류

```
/claude-discode:km lite     # 개인 vault
/claude-discode:km at       # 팀 vault (병렬)
```

10.5. 시나리오 5: 회의 시작

상황 — 팀 회의 시작 시 회의록 폴더 + agenda template 자동 셋업

```
/claude-discode:meetings new spec-review
```

10.6. 시나리오 6: Codex second opinion

상황 — 카파시 (Claude) 가 architecture 결정 unsure → Codex (GPT-5.5) 의견 수렴

```
/codex YAML schema 설계 의견 요청
```

11. v1.0 Custom Hybrid YAML (새 agent 추가)

claude-discode 의 모든 skill + command 는 `.agents/<name>.yaml` 1개 파일로 정의됨. 학생도 본인 agent 신설 가능.

11.1. 5 Block 구조

```yaml
# Block A — agentskills.io base (portable)
name: my-custom-agent
description: 내 커스텀 agent
version: 1.0.0
license: MIT

# Block B — Hermes provides_* (lifecycle)
provides_tools: [my_tool]
provides_commands: [/my-cmd]

# Block C — claude-discode extension (classroom)
tier: external           # external | internal | experimental | deprecated
model: opus[1m]
allowedTools: [Bash, Read, Glob]

# Block D — Dynamic gates (verification)
gates:
  - id: latency-check
    expr: "latency_ms_p95 < 2000"
    evaluator: simple    # simple | cel

# Block E — Benchmark integration
benchmark:
  fixtures: benchmark/fixtures/queries.yaml
  axes: [latency_ms, recall_at_k]
  baseline_tier: 4
```

11.2. 추가 가이드

1. `.agents/<your-agent>.yaml` 신규 생성
2. `npm run validate` — JSON Schema 통과 확인
3. `npm run index` — agents.yaml index 갱신
4. PR 생성 — GitHub Actions 가 자동 검증

11.3. Tier 의미

tier	의미	학생 노출
external	강의 / 외부 사용자 안전	✅
internal	운영자 전용	부분
experimental	PoC / 미검증	❌
deprecated	사용 중단 예정 (6 month timeline)	⚠️

12. v1.1 graduate (강의 후 evolution)

12.1. 왜 v1.0 single-shot 인가?

GPT-5.5 가 "표준 너무 일찍 고정 시 obsolete risk" 경고. 그러나 사용자 결정 = "강의 deadline 우선, 강의 후 v1.1 graduate" → Single-shot v1.0 cut.

12.2. Graduate trigger 3 조건

`benchmark/results/*.json` 3주 연속 다음 중 1개 발생 시 v1.1 PR 자동:

1. 어떤 metric 이든 ±20% 이상 drift
2. 강의 수강생 feedback (GitHub Discussions) "재현 성공률 ⚠️/❌" 3건 이상
3. `tier1-stale > 14 days` badge 14일 이상 지속

12.3. v1.0 → v1.1 backward compat

`scripts/agents-migrate.mjs` 자동 upgrade:

- deprecated field 변환
- 새 field default 채움
- CHANGELOG.md 에 breaking changes badge

12.4. 강의 1차 코호트 feedback

GitHub Discussions Feedback category 5 질문 schema:

1. Setup 완료 시간 (분)
2. Tier 1 GraphRAG 재현 성공률 (✅ / ⚠️ / ❌)
3. 가장 답답했던 부분 (자유 기술)
4. CEL 이해도 (1-5점)
5. v1.1 우선순위 TOP 3 제안

12.5. Timeline

```
2026-05-13   v1.0 release (오늘)
2026-05-13~27 2주 lock — 강의 1차 코호트 dogfood
2026-05-28~  v1.1 graduate decision (drift + feedback 분석)
2026-06-04   v1.1 PR 머지 + vault 측 mirror sync 시작
```

13. FAQ + 도움

13.1. Q1. macOS / Linux / Windows / WSL 다 됩니까?

- macOS ✅ 검증 완료
- Linux ✅ 검증 완료
- WSL ✅ 검증 완료
- Windows native — 추후 지원 (현재 WSL 권장)

13.2. Q2. 학생인데 비용 걱정?

- Tier 4 (ripgrep) + Tier 2 (Obsidian) = 100% 무료
- Tier 3 (MCP) = 무료 (Claude Code 구독 안에)
- Tier 1 (GraphRAG) = OpenAI/Anthropic API → 본인 vault 크기에 따라 1회 indexing $0.5~5

13.3. Q3. 이미 obsidian-cli 만 쓰고 있는데 굳이 claude-discode 까지?

obsidian-cli 만 사용 → recall ≈ 0.62 claude-discode 4-Tier → recall ≈ 0.85

+37% recall 차이가 본인 use case 에 의미 있다면 도입 가치 있음. 의미 없으면 obsidian-cli 만으로도 OK.

13.4. Q4. 셋업 도중 막혔어요

```
cat ~/.claude-discode-setup.log
```

GitHub Issue 등록 (자동 redaction 적용된 log):

https://github.com/treylom/claude-discode/issues/new?template=setup-failure.yml

13.5. Q5. 새 agent / skill 추가하고 싶어요

본 매뉴얼 Section 11 (Custom Hybrid YAML) 참고. `.agents/<name>.yaml` 1개 파일 + JSON Schema 검증 + PR.

13.6. Q6. 강의 외 일반 사용자도 환영?

네. claude-discode 는 MIT license. 본인 vault 운영, 팀 KM, Discord 봇 운영 어디든 자유 사용.

13.7. Q7. 피드백 어디 남기나요?

GitHub Discussions Feedback category https://github.com/treylom/claude-discode/discussions/categories/feedback

5 질문 schema 로 2분 응답 → v1.1 graduate decision 에 반영

도움이 필요하면

- GitHub Issue: https://github.com/treylom/claude-discode/issues
- GitHub Discussions: https://github.com/treylom/claude-discode/discussions
- 강의 학생: 강의 Discord 채널
- 관련 문서: SETUP.md (개발자) / AGENTS.md (Custom Hybrid v1.0) / BENCHMARK.md (5-axis)

claude-discode v1.0.0 · 2026-05-13 · MIT